

DDL

CREATE, ALTER en DROP

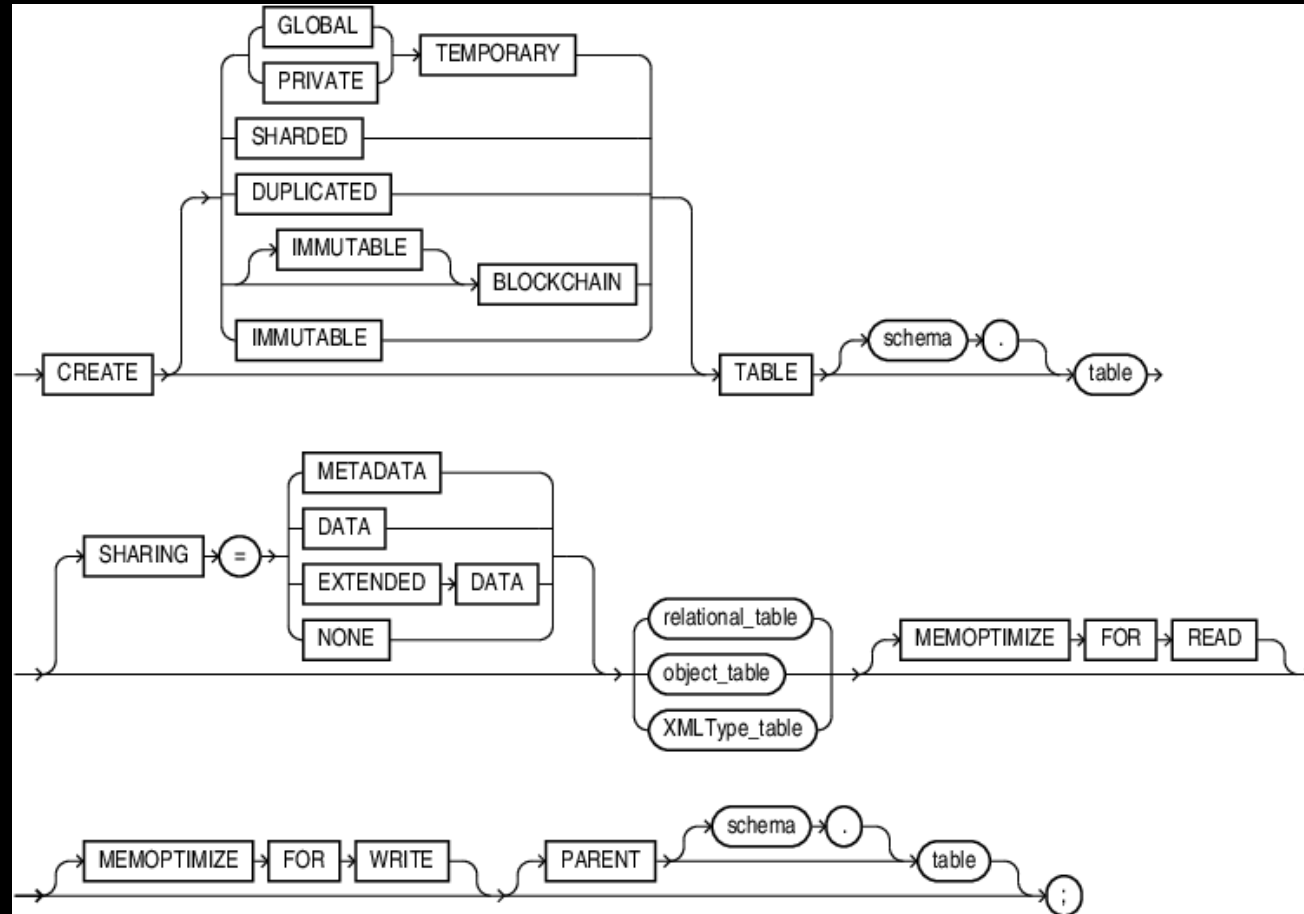
Aanmaken van tabellen

CREATE TABLE

CREATE TABLE

- CREATE TABLE wordt gebruikt om tabellen aan te maken

CREATE TABLE syntax



CREATE TABLE syntax

```
CREATE TABLE tabelnaam (  
    kolomnaam kolomtype [NOT NULL | NULL] [DEFAULT]  
    [, kolomnaam kolomtype [NOT NULL | NULL] [DEFAULT]]  
    [CONSTRAINT naam  
        PRIMARY KEY (kolomnaam [, kolomnaam]...)]  
    [CONSTRAINT naam  
        FOREIGN KEY (kolomnaam [,kolomnaam]...)  
        REFERENCES tabelnaam (kolomnaam [, kolomnaam]...)]  
);
```

Voorbeeld

```
CREATE TABLE werknemers (  
    wnr      NUMBER(3),  
    wnaam    VARCHAR2(25),  
    afdeling CHAR(2),  
    fnaam    VARCHAR2(18),  
    salaris  NUMBER(7,2),  
    vesnr    NUMBER(3)  
);
```

Primary & foreign keys

- PRIMARY KEY definieert de primaire sleutel (identifieer in het ERD)
- FOREIGN KEY legt relatie vast tussen twee tabellen → integriteit!
- Definiëren aan de hand van constraints

Constraints

- Dwingt bepaalde zaken af voor een kolom
- Primary key constraint
 - Waarden moeten uniek zijn
 - Mag niet NULL zijn
- Foreign key constraint
 - Waarde moet verwijzen naar een bestaande waarde in een (andere) tabel

Primary & foreign keys

```
CREATE TABLE vestigingen (  
    id          NUMBER(3),  
    vesnaam     VARCHAR2(20),  
    CONSTRAINT pk_vestigingen_id PRIMARY KEY(id));
```

```
CREATE TABLE werknemers (  
    id          NUMBER(3),  
    wnaam       VARCHAR2(255),  
    vesnr       NUMBER(3),  
    CONSTRAINT pk_werknemers_id PRIMARY KEY(id),  
    CONSTRAINT fk_werknemers_vestigingen_vesnr  
        FOREIGN KEY(vesnr) REFERENCES vestigingen(id));
```

NOT NULL constraint

- Bepaalt optionaliteit
- Duidt aan of de waarde verplicht is bij invoeren (INSERT)
- Standaard: NULL

NOT NULL | NULL

```
CREATE TABLE werknemers
(
    id NUMBER(3),
    wnaam VARCHAR2(255) NOT NULL,
    salaris NUMBER(7, 2) NULL,
    fnaam VARCHAR2(18),
    CONSTRAINT pk_werknemers_id PRIMARY KEY(id)
);
```

UNIQUE constraint

- Zorgt dat de waarden van een kolom uniek zijn
- PRIMARY KEY krijgt automatisch UNIQUE en NOT NULL constraints

UNIQUE

```
CREATE TABLE werknemers
(
    id NUMBER(3),
    wnaam VARCHAR2(255) UNIQUE,
    email VARCHAR2(255) UNIQUE NOT NULL,
    salaris NUMBER(7, 2) NULL,
    fnaam VARCHAR2(18),
    CONSTRAINT pk_werknemers_id PRIMARY KEY(id)
);
```

Extra: CHECK constraint

- Logische controles
- Meer info: Data Advanced

[illegible]

Extra: DEFAULT

- Zet standaardwaarde als er geen waarde is ingevuld bij INSERT

Extra: DEFAULT

```
CREATE TABLE werknemers
(
    id NUMBER(3),
    wnaam VARCHAR2(255) NOT NULL,
    salaris NUMBER(7, 2) DEFAULT 1000,
    fnaam VARCHAR2(18) DEFAULT 'ANALIST',
    CONSTRAINT pk_werknemers_id PRIMARY KEY(id)
);
```


Courante datatypes

- CHAR, VARCHAR2
- NUMBER
- DATE, TIMESTAMP
- RAW, BLOB, CLOB
- XML (SQL:2003)
- JSON (SQL:2016)

Precisie van datatypes

- Bepaalt de lengte van de waarden in de kolom
- *kolomnaam* DATATYPE(grootte)

Precisie van datatypes

- wnaam VARCHAR2(15)
 - wnaam mag maximum 15 karakters lang zijn
 - 'Wim Hambrouck' → OK
 - 'Wim Met Lange Achternaam' → ERROR: value too large for column
- leeftijd NUMBER(2)
 - leeftijd mag maximum 2 cijfers lang zijn
 - [-99, 99] → OK
 - 150 → ERROR: value larger than specified precision allowed for this column

NUMBER

- Gehele of decimale getallen
- `NUMBER[(p[,s])]`
 - p = precisie (totaal aantal cijfers)
 - s = scale = aantal decimalen (cijfers na de komma)
- P is getal tussen 1 en 38
- S is getal tussen -84 en 127

NUMBER

- NUMBER(5)
 - Lengte van 5 cijfers, geen beduidende cijfers
 - [-99999, 99999]
→ OK
 - 12345.12
→ Geen error, maar getal wordt wiskundig afgerond! (12345)
 - 123456
→ ERROR: value larger than specified precision allowed for this column

NUMBER

- NUMBER(5, 2)
 - Lengte van 5 cijfers, waarvan 2 beduidende cijfers
 - 123 → OK (123.00)
 - 123.45 → OK (123.45)
 - 1234.5 → ERROR (1234.50)
 - 1234 → ERROR (1234.00)
 - 123.456 → OK, maar afronding (123.46)

DATE

- Datum en tijd
- Opslag van datums tussen 1 januari 4712 BCE¹ tot en met 31 december 9999
- Tijd wordt ook opgeslagen, maar standaard niet zichtbaar
- Vaste precisie (tot op de seconde nauwkeurig)

¹ https://en.wikipedia.org/wiki/Common_Era

DATE

- Standaardformaat DD-MON-YY of DD-MON-YYYY
 - '01-NOV-99' = 1 november 1999 00:00:00
 - '25-NOV-2099' = 25 november 2099 00:00:00
- Onvolledige of ongeldige invoer geeft ERROR
- SYSDATE geeft huidige datum en tijd terug

Extra: TIMESTAMP

- Datum en tijd met extra precisie
 - DATE heeft vaste precisie (tijd met geheel aantal seconden)
 - TIMESTAMP heeft seconden met cijfers na de komma
- Voor het opslaan van nauwkeurige tijdwaarden en voor het opslaan en vergelijken van datumgegevens uit verschillende tijdzones

Extra: TIMESTAMP

- `TIMESTAMP[(M)]`
 - M = aantal cijfers na de komma
 - Standaard: 6
 - Minimum: 0
 - Maximum: 9

CHAR

- Alfnumerieke waarden met vaste lengte
- CHAR[(M)]
 - Standaard: 1
 - Maximum: 2000
- Waarden korter dan de maximumlengte worden aangevuld met spaties

CHAR

- CHAR(3)
 - 'ABC' → 'ABC'
 - 'A' → 'A__'
 - 'AB' → 'AB_'
 - '' → NULL
 - 'ABCD' → ERROR: value too large for column

VARCHAR2

- Alfnumerieke waarden met variabele lengte
- VARCHAR2(M)
 - Geen standaardlengte, altijd zelf opgeven
 - Minimum: 1
 - Maximum: 4000

VARCHAR2

- VARCHAR2(3)
 - 'ABC' → 'ABC'
 - 'A' → 'A'
 - 'AB' → 'AB'
 - '' → NULL
 - 'ABCD' → ERROR: value too large for column

Waarom VARCHAR2?

- Enkel in Oracle Database (andere RDBMS gebruiken VARCHAR)
- “Do not use the VARCHAR data type. Use the VARCHAR2 data type instead. Although the VARCHAR data type is currently synonymous with VARCHAR2, the VARCHAR data type is scheduled to be redefined as a separate data type used for variable-length character strings compared with different comparison semantics.”

CLOB

- Character Large Object
- Heel lange tekst, tot ongeveer 32 terabyte aan karakters
- Trager dan CHAR of VARCHAR

BLOB

- Binary Large Object
- Binaire data (bestanden), tot ongeveer 32TB
- Relatief traag bij opzoeken
- Best practice: in aparte tabel
- Alternatief voor (kleine) binaire data: RAW met precisie

Andere datatypes

- Veel RDBMS hebben nog specifiekere datatypes
- Vaak implementaties van bestaande datatypes uit de SQL-standaard
 - TINYTEXT = VARCHAR2(255)
 - BOOLEAN = NUMBER(1)
 - FLOAT = NUMBER(38, 126)

Extra: IDENTITY

- Automatisch genereren van PRIMARY KEY

```
CREATE TABLE werknemers  
(  
    id NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY,  
    wnaam VARCHAR2(255) NOT NULL,  
    CONSTRAINT pk_werknemers_id PRIMARY KEY(id)  
);
```

- Niet alle RDBMS houden zich hiervoor aan de SQL-standaard

Meer info

- <https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/Data-Types.html#GUID-7B72E154-677A-4342-A1EA-C74C1EA928E6>

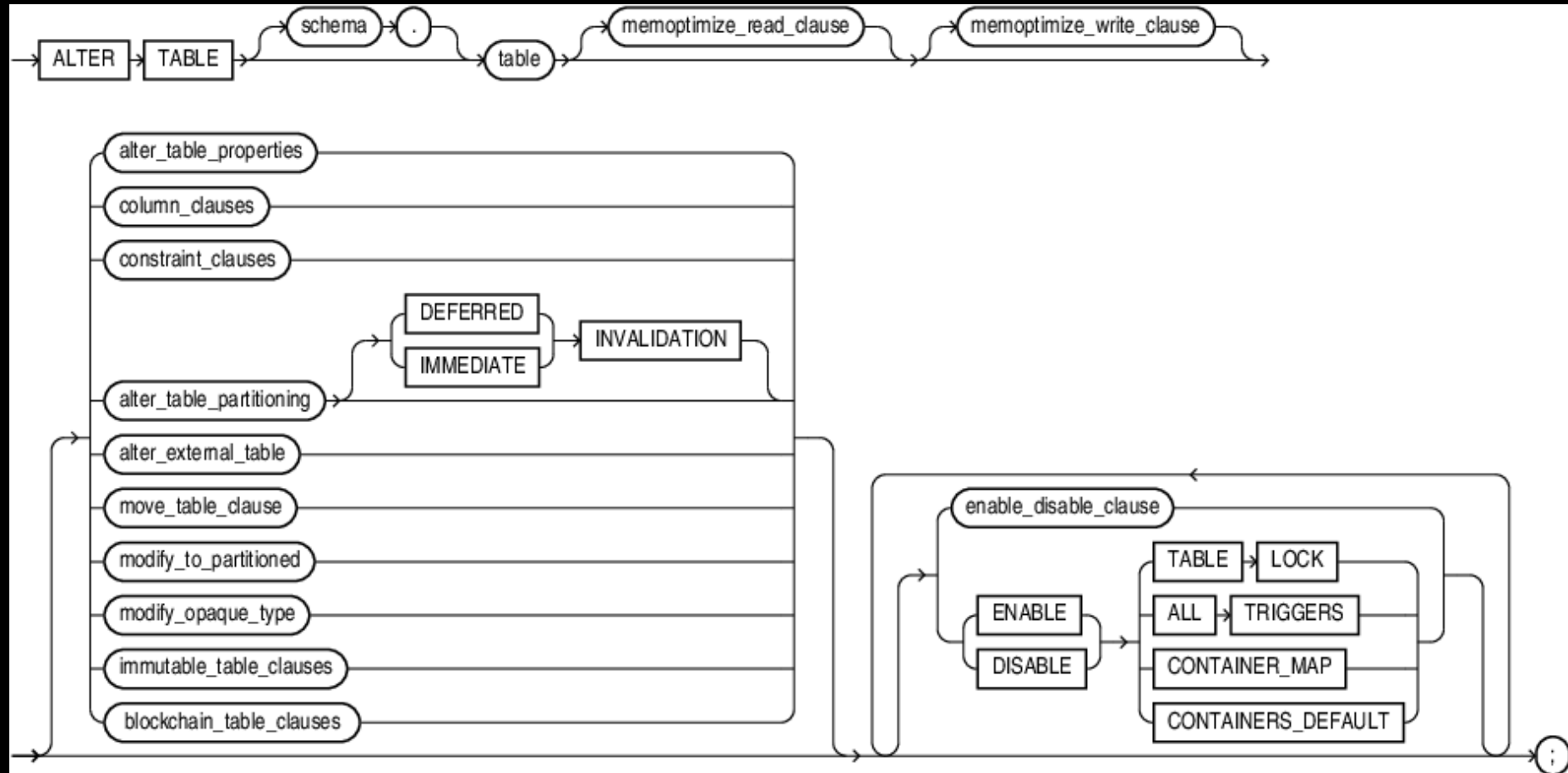
Wijzigen van tabellen

ALTER TABLE

ALTER TABLE

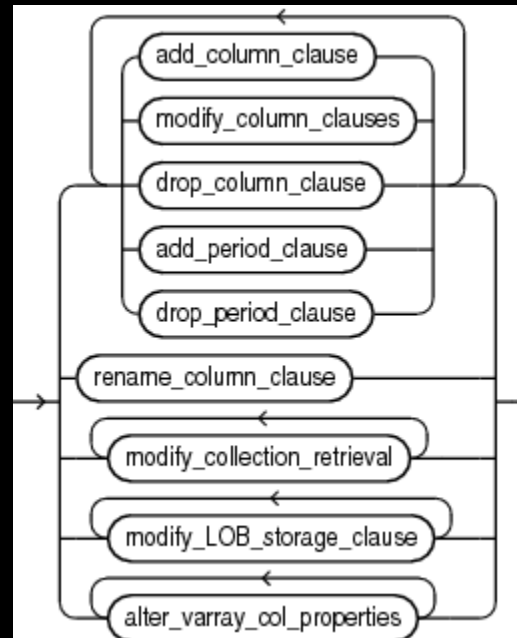
- ALTER TABLE wordt gebruikt om de structuur van een tabel veranderen na aanmaken
- Nuttig als er al data in de tabel zit

ALTER TABLE syntax



ALTER TABLE syntax

- column_clauses



Kolom toevoegen

- Voeg een kolom achternaam toe aan de tabel werknemers

```
ALTER TABLE werknemers  
ADD achternaam VARCHAR2(10);
```

Kolom aanpassen

- Pas de kolom achternaam aan in de tabel werknemers zodat deze maximum 255 karakters kan bevatten

```
ALTER TABLE werknemers  
MODIFY achternaam VARCHAR2(255);
```

Kolom aanpassen

- Pas de kolom achternaam aan in de tabel werknemers zodat deze verplicht is
- Kan enkel als er voor achternaam nog geen NULL-waarden in de tabel zitten

```
ALTER TABLE werknemers  
MODIFY achternaam VARCHAR2(255) NOT NULL;
```

Verplichte kolommen (NOT NULL)

- Kolommen aanpassen naar NOT NULL kan enkel als de kolom nog geen NULL-waarden bevat
- Nieuwe kolommen bijvoegen met NOT NULL kan niet
 - Optie 1: nieuwe kolom toevoegen, data invoegen en dan wijzigen naar NOT NULL
 - Optie 2: een DEFAULT waarde voorzien

Kolom toevoegen met NOT NULL: Optie 1

```
ALTER TABLE werknemers  
ADD achternaam VARCHAR2(255);
```

```
UPDATE werknemers  
SET achternaam = 'onbekend';
```

```
ALTER TABLE werknemers  
MODIFY achternaam VARCHAR2(255) NOT NULL;
```

Kolom toevoegen met NOT NULL: Optie 2

```
ALTER TABLE werknemers  
ADD achternaam VARCHAR2(255) NOT NULL DEFAULT 'Onbekend';
```

Kolom verwijderen

- Verwijder de kolom achternaam uit de tabel werknemers

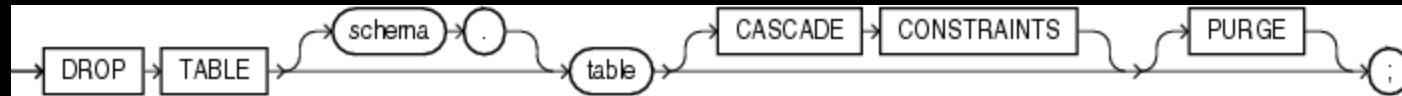
```
ALTER TABLE werknemers  
DROP COLUMN achternaam;
```

Verwijderen van tabellen

DROP TABLE

DROP TABLE

- DROP TABLE vernietigt de volledige tabel en alle data die er in zit



DROP TABLE syntax

- DROP TABLE *tabelnaam*;
- Gebeurt onmiddellijk
- Niet terug te draaien met ROLLBACK (zie TCL)

Voorbeeld

```
DROP TABLE werknemers;
```

Opgelet met foreign keys

- Als er andere tabellen zijn met foreign key(s) die verwijzen naar de tabel die wordt verwijderd, zal het verwijderen niet doorgaan
 - ERROR: unique/primary keys in table referenced by foreign keys
- Oplossing: eerst andere tabel(len) verwijderen
- Andere optie: CASCADE CONSTRAINTS toevoegen
 - **OPGELET: Dit is ten zeerste af te raden, want zorgt voor data-inconsistentie!**